Indian Institute of Technology Bombay

IE 624: Generative and Agentic AI

Report On

Reproducing and Stress-Testing ICON:

(Inference-Time Correction for Indirect Prompt Injection Defense)

Authors: Mohammad Kadiri (210040142) | Chaitanya Katii (210100044) | Gaurav Kumar (23B2715)

Abstract

LLM agents that operate in tool-augmented settings are vulnerable to Indirect Prompt Injection (IPI) attacks, where malicious instructions hidden inside retrieved content redirect the agent toward attacker-specified goals without any user interaction. ICON [1] proposes an inference-time defense built on the observation that IPI attacks leave a measurable attention collapse signature in the model's latent space. It detects this signature using a lightweight probe trained on a small number of samples, and neutralizes it by steering attention weights before the softmax step, which lets the agent continue its original task rather than aborting. In this project we pursue two goals. First, we reproduce the three core components of ICON, namely the Focus Intensity Score (FIS), the Latent Space Trace Prober (LSTP), and the Mitigating Rectifier (MR), using publicly available backbone models. Second, because the TrojanTools training corpus [5] is not yet publicly released, we implement the paper's own LLM-as-Optimizer attack generation loop to synthesize a surrogate dataset. Third, we design and evaluate four attack strategies that exploit structural assumptions in ICON's detection mechanism and measure how much each one degrades its performance. Experiments will be conducted on the publicly available InjectAgent benchmark [2], and results will be reported in terms of Attack Success Rate (ASR) and Utility under Attack (UA), consistent with the original paper.

Contents

1 Introduction

LLM-based agents connected to external tools, web browsers, and APIs can now carry out complex multi-step tasks on a user's behalf. This autonomy creates an attack surface that is fundamentally different from conventional adversarial inputs. In an IPI attack, a malicious instruction is embedded inside a document or API response that the agent retrieves during normal operation, for example inside an email body or a search result. The agent then processes this content as data but inadvertently follows it as an instruction, causing it to call unauthorized tools or leak private information.

Existing defenses have not solved this problem adequately. Prompt-level strategies such as delimiter injection and system prompt repetition rely on patterns that adaptive attackers can bypass with minor rewording. Tool-filtering approaches such as MELON [4] block suspicious tool calls after they are generated, but they cannot restore the agent's reasoning state once a filter fires, so task utility is permanently lost. Safety fine-tuning methods achieve stronger security but require millions of training examples and many hours of compute, and they tend to refuse too aggressively, interrupting valid multi-step workflows.

ICON [1] takes a different approach. Its core observation is that a successful IPI attack forces the model to concentrate attention on a small number of injected tokens rather than distributing it across the full reasoning context. This concentration is measurable as a low-entropy (high focus intensity) signature in specific attention heads. A lightweight probe trained on only 255 samples can detect this signature reliably, and a targeted attention steering operation can suppress it before it changes the model's output, letting the agent complete its original task. The paper reports an average ASR of 0.4% and a utility improvement of over 50% compared to existing baselines, with a training time under two minutes.

Despite these results, the paper does not analyze what kinds of attacks can evade ICON's detection. The threat model assumes a grey-box adversary with no access to the model's internal attention weights. A stronger adversary who knows ICON is deployed could craft payloads that produce benign-looking attention patterns while still hijacking the agent. This gap motivates the stress-testing component of our project.

Contributions.

1. We implement FIS, LSTP, and MR in a self-contained PyTorch codebase and validate the attention collapse hypothesis on the InjectAgent benchmark.
2. We replicate the paper's adaptive attack generation pipeline using an LLM-as-Optimizer loop, producing a surrogate training corpus that substitutes for the unreleased TrojanTools dataset.
3. We design and empirically evaluate four attack strategies that target ICON's detection mechanism and measure how much each one degrades its performance.

2 Background

2.1 Indirect Prompt Injection

Following the formalization in [1], a tool-augmented agent $\mathcal{M}$ operates over a trajectory $\mathcal{S}_t = (I_u, \mathcal{A}_{1:t-1}, \mathcal{O}_{1:t-1})$, where I_u is the user instruction, $\mathcal{A}_{1:t-1}$ is the sequence of past tool calls, and $\mathcal{O}_{1:t-1}$ is the sequence of past observations including retrieved content. An IPI attack injects a payload δ into the observation stream so that:

$$\mathcal{M}(\mathcal{S}_t \oplus \delta) = f_{\text{adv}}, \tag{1}$$

where f_{adv} is an unauthorized tool call and $\oplus$ denotes the injection operation. Adaptive attacks additionally require that δ be semantically consistent with the surrounding benign content so that surface-level filters cannot distinguish it.

2.2 ICON's Three Core Components

2.2.1 Focus Intensity Score

Let $\mathbf{A}_{l,h} \in \mathbb{R}^{N_Y \times N}$ be the attention weight matrix of head h in layer l, where N_Y is the number of generated tokens and N is the context length. ICON computes the generation-normalized entropy for each query token i:

$$E(l,h,i) = -\frac{1}{\log N} \sum_{j=1}^{N} a_{i,j} \log(a_{i,j} + \varepsilon). \tag{2}$$

The Focus Intensity Score for head h in layer l is the complement of the mean entropy across the generated sequence:

$$S_{l,h} = 1 - \frac{1}{N_Y} \sum_{i=1}^{N_Y} E(l,h,i). \tag{3}$$

A high value of $S_{l,h}$ indicates that head h is concentrating its attention on a small number of tokens, which is the signature ICON uses for detection.

2.2.2 Latent Space Trace Prober

For each head, ICON extracts a three-dimensional descriptor:

$$\mathbf{v}_{l,h} = \left[\min_i E_{l,h,i}, \quad \frac{1}{N_Y} \sum_{i=1}^{N_Y} E_{l,h,i}, \quad \sqrt{\frac{1}{N_Y} \sum_{i=1}^{N_Y} (E_{l,h,i} - \bar{E}_{l,h})^2} \right] \in \mathbb{R}^3. \tag{4}$$

Descriptors from the top K layers ranked by FIS are concatenated into a global feature vector $\mathbf{z} \in \mathbb{R}^{3 \times H \times K}$, for example $\mathbb{R}^{384}$ when $H = 32$ and $K = 4$. This vector is fed to a hybrid 1D-CNN and MLP binary classifier with approximately 31,000 parameters.

2.2.3 Mitigating Rectifier

When the prober flags an input as an attack, the MR modifies the raw attention logits of the flagged heads before softmax is applied. A suppression threshold $\theta_{l,h}$ is set at the $(1-\tau)$-th percentile of the raw attention values in head h (scope parameter τ). The binary steering mask is:

$$M_{l,h}(i,j) = \mathbb{I}[a_{l,h}(i,j) \geq \theta_{l,h}]\,. \tag{5}$$

Attention values are then scaled by intensity parameter $\gamma < 1$:

$$\tilde{A}_{l,h} = A_{l,h} \odot \big[1 + M_{l,h} \cdot (\gamma - 1)\big]. \tag{6}$$

Because softmax is applied after this step, suppressing the peak values naturally redistributes probability mass toward benign context tokens, restoring the agent's original attention distribution without modifying any model weights.

3 Proposed Methodology

3.1 Model and Infrastructure

We use QWEN-3-8B as the primary backbone, loaded in 4-bit NF4 quantization via `bitsandbytes` to keep the VRAM requirement at around 6 to 8 GB. We disable Flash Attention by setting `attn_implementation="eager"` so that intermediate attention logits are accessible before the softmax step. Attention matrices are captured by registering PyTorch forward hooks on each `self_attn` module during `model.generate()`. The agent runs in a ReAct loop with a small set of mock tools (for example `send_email`, `search_web`, and `make_payment`) to simulate realistic agentic scenarios.

3.2 Adaptive Attack Data Synthesis

TrojanTools [5] is currently under review and has not been released. We therefore implement Section 4.3 of ICON directly to produce a surrogate dataset of approximately 255 labeled samples.

The generation procedure solves the attacker's objective from the paper:

$$\min_{\delta}\ \mathcal{D}(\mathcal{S}_t \oplus \delta,\, \mathcal{S}_t) \quad \text{subject to} \quad \mathcal{M}(\mathcal{S}_t \oplus \delta) = f_{\text{adv}}, \tag{7}$$

where $\mathcal{D}$ measures semantic discrepancy between the injected and benign trajectories.

The procedure has two steps. First, for each benign task scenario drawn from InjectAgent, a capable LLM acting as the optimizer (we use GPT-4o via API) selects the most contextually plausible adversarial tool f_{adv} from the available tool set and generates an initial injection δ. Second, the optimizer receives feedback on whether the attack succeeded and how semantically consistent δ is with the surrounding observation, then rewrites δ to improve on both dimensions. This loop runs for up to five rounds per scenario. Trajectories that result in successful and stealthy attacks are labeled ATTACK; the original unmodified trajectories are labeled BENIGN.

3.3 ICON Replication

We implement all three components in PyTorch.

FIS is computed at every generation step by passing collected attention matrices through Equations 2 and 3. The top $K = 4$ layers by mean FIS differential between attack and benign samples are selected as attack-sensitive layers.

LSTP is a 1D-CNN with two convolutional layers followed by adaptive max pooling and a two-layer MLP. It is trained on the synthesized dataset using binary cross-entropy loss.

MR is implemented by monkey-patching the `forward` method of the target attention layers to intercept raw QK dot-product scores, apply the mask in Equation 5, scale by Equation 6, and then pass the modified scores to the standard softmax. The hook is activated only when the LSTP flags a trajectory as an attack. Hyperparameters γ and τ are selected by grid search on a held-out validation split of the synthesized dataset.

3.4 Adversarial Stress-Testing

We evaluate the conditions under which ICON fails to detect an IPI attack. We design four attacks, each targeting a different assumption in ICON's detection mechanism.

Attack 1: FIS-Aware Latent Mimicry. We add a third constraint to the synthesis loop: the FIS score of the injected trajectory must fall below the prober's detection threshold. The modified objective is:

$$\min_{\delta} \mathcal{D}(\mathcal{S}_t \oplus \delta, \mathcal{S}_t) + \lambda \cdot \mathrm{FIS}(\mathcal{S}_t \oplus \delta) \quad \text{subject to} \quad \mathcal{M}(\mathcal{S}_t \oplus \delta) = f_{\text{adv}}. \tag{8}$$

This attack directly optimizes against ICON's detection signal and represents the strongest adaptive threat we evaluate.

Attack 2: Long-Context Attention Dilution. The injection payload is wrapped inside a long block of benign text. We test padding lengths of 100, 300, 500, and 1000 tokens and measure how the additional context distributes attention across more tokens, reducing the per-token FIS spike below the detection threshold. We report ASR as a function of surrounding context length.

Attack 3: Gradual Multi-Step Injection. The malicious intent is split across two consecutive observation steps. The first step plants a benign-looking precondition and the second step activates it. Because ICON evaluates FIS independently at each generation step, neither step triggers detection on its own, while the two steps together achieve the adversarial goal.

Attack 4: Attention Sink Routing. Autoregressive transformers consistently route a portion of attention to a small set of tokens regardless of content, commonly referred to as attention sinks [6]. We craft injections designed to route their semantic influence through these sink tokens rather than concentrating it directly on injection tokens, with the goal of hiding the attention collapse signature from FIS.

4 Evaluation Protocol

All experiments use the InjectAgent benchmark [2] under three attack conditions: Ignore Instruction (the simplest attack template), Combined Attacks (multiple templates mixed), and our synthesized adaptive attacks.

We try to report the following metrics in line with the original paper:

- **ASR (Attack Success Rate, lower is better):** the fraction of attack scenarios in which the agent executes f_{adv}.
- **UA (Utility under Attack, higher is better):** the fraction of the original task objectives that the agent completes despite the attack.
- **ADR (Attack Detection Rate, higher is better):** the fraction of attacks correctly flagged by the LSTP prober.
- **URR (Utility Recovery Rate, higher is better):** the fraction of utility that the MR recovers relative to the no-defense baseline.

Baselines are No Defense, Repeat Prompt [3], Delimiting, and MELON [4].

5 Expected Outcomes

We expect to produce the following:

- A working PyTorch re-implementation of FIS, LSTP, and MR that reproduces ICON's core result on InjectAgent.
- A synthesized IPI dataset of approximately 255 samples built using the paper's own generation algorithm.
- Empirical confirmation that attention collapse is a reliable detection signal for standard IPI attack templates.

- Quantitative results for four breaking attacks, including ASR measurements at each attack configuration and an analysis of why each attack succeeds or fails against ICON.

6 Conclusion

This project reproduces and stress-tests ICON, a recent inference-time defense against indirect prompt injection in LLM agents. We implement the full pipeline, generate our own training data using the paper's attack synthesis algorithm, and evaluate four targeted attacks that exploit structural assumptions in ICON's detection mechanism. The reproduction validates the attention collapse hypothesis on a publicly available benchmark, and the stress-testing provides a quantitative picture of where the defense breaks down.

References

[1] Wang, C., Zhang, F., Zhang, J., Zhang, Z., Wang, Y., Huang, L., Gao, J., Chen, Z., and Lim, W. Y. B. "ICON: Indirect prompt injection defense for agents based on inference-time correction." *arXiv preprint arXiv:2602.20708*, 2026.

[2] Zhan, Q., Liang, Z., Ying, Z., and Kang, D. "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents." In *Findings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 10471–10506, 2024.

[3] Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., and Tramer, F. "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents." In *NeurIPS Datasets and Benchmarks Track*, 2024.

[4] Zhu, K., Yang, X., Wang, J., Guo, W., and Wang, W. Y. "MELON: Provable defense against indirect prompt injection attacks in AI agents." *arXiv preprint arXiv:2502.05174*, 2025.

[5] Anonymous. "TrojanTools: Adaptive indirect prompt injection on LLM agents via malicious tool-calling." Submitted to the Fourteenth International Conference on Learning Representations, 2025. Under review.

[6] Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. "Efficient streaming language models with attention sinks." In *The Twelfth International Conference on Learning Representations*, 2024.